Lesson 10 Teacher's Key: Play It Again

Scratch - Instructor Reference Only

VIDEO GAME CREATION

What This Lesson Teaches

The clearest possible version of this lesson's core idea: a broadcast is Scratch's natural 'reusable routine' - the same broadcast can be sent from the green flag AND from a key-press script, and every receiving script runs identically either way.

Level 1 - Full Worked Solution

- The refactor is almost entirely mechanical: what used to run directly under "when green flag clicked" now runs under "when I receive [start game]" instead, with the green flag script reduced to a single broadcast.
- The freeze is one new block at the top of the clone's existing forever loop: "if game_active = false then stop this script". It is an early exit, the same shape as Godot's early return, GameMaker's exit and Unity's early return, and it needs no re-indenting of the existing blocks. (Credit: Robert's own testing improved on the original wrap-it-in-an-if design.) A finished round freezes the enemies in place, matching the other engines.
- game_active is flipped to false at the exact moment score/lives would have crossed to a win/lose state, right alongside the existing broadcast - one new line in an already-correct location, twice (once per branch).

Highlighted lines are new this week; everything else is code you already have.

```
// GameManager (hidden sprite):
when green flag clicked
hide
broadcast [start game]

when I receive [start game]
set score to 0
set lives to 3
set enemies_remaining to 5
set game_active to true

when I receive [enemy hit]
change enemies_remaining by -1
change score by 10
if enemies_remaining = 0 then
    set game_active to false
    broadcast [you win]

when I receive [enemy escaped]
```

```
change lives by -1
if lives = 0 then
    set game_active to false
    broadcast [game over]
else
    start sound [sfx_lifelost]  // your Lesson 8 Level 3 sound

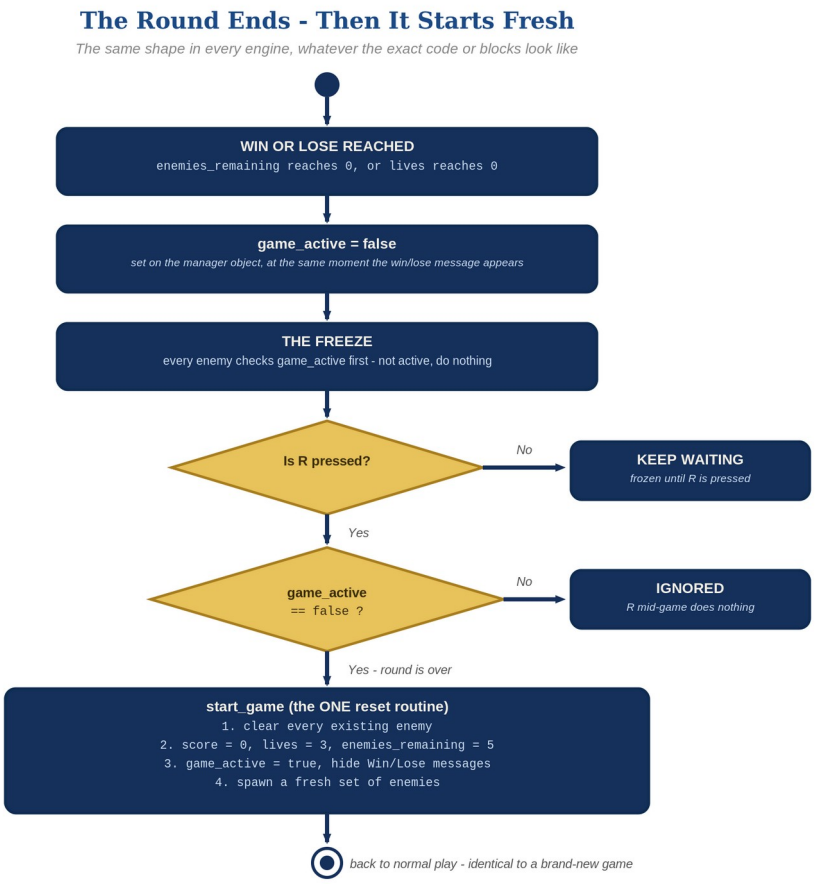
when key r pressed
if game_active = false then
    broadcast [start game]

// Enemy sprite:
when I receive [start game]
repeat 5
    create clone of myself
    // (your existing position and wait blocks stay here)

when I receive [start game]
delete this clone

when I start as a clone
forever
    if game_active = false then
        stop this script
    // (your existing movement blocks stay here)
    if touching Bullet? then
        // (your existing hit logic stays here)
    if y position < -170 then
        // (your existing escape logic stays here)

// Player sprite:
when I receive [you win]
start sound [sfx_win]  // your Lesson 6 Level 3 sound
say [You Win! Press R to Restart] for 3 seconds
when I receive [game over]
start sound [sfx_gameover]  // your Lesson 8 Level 3 sound
say [Game Over - Press R to Restart] for 3 seconds
```



When the round ends, everything freezes - and R, pressed only while the round is over, runs the one reset routine. The same shape in every engine.

Why This Structure

- Broadcasts were already the tool Lesson 6 used to announce a win - this lesson simply reveals that the exact same tool can also be the entry point for a full game reset, with zero new Scratch concepts required.
- The clean-up is a SECOND "when I receive [start game]" script on the Enemy sprite holding only "delete this clone" (an end block, so it cannot sit above the spawn loop). Clones hear broadcasts too: every old clone deletes itself, while the original sprite ignores the block and its own spawn script makes the new set. Without it, surviving clones would also run the spawn loop on restart.

Level 2 / Level 3 Talking Points

- Level 3 bug hunt: kills reduce enemies_remaining but escapes only reduce lives, so a round with some kills and some escapes (for example 3 and 2) ends with no enemies left, no win and no loss, and R does nothing because game_active is still true. This started in Lesson 6 (win = all killed) and Lesson 8 (lose = lives gone). A good fix, and the one shown here, is that an escape also resolves an enemy and the win check requires

lives above 0. Accept any design that guarantees every round ends in exactly one outcome (for example a third 'Round Over' state, or a check that no enemies remain and none are in flight). Test it with 3 kills and 2 escapes, then 4 and 1, then 0 and 5.

- Ask students to trace, out loud, everything that happens when they press R while game_active is false - broadcast sent, Enemy's "when I receive" script runs, the clones run their delete script while the original sprite spawns the fresh five. A good comprehension check.
- For strong students: ask what would happen if the green-flag script were left as a full copy of the setup code (not refactored into a broadcast) alongside a separate restart script - the answer (two copies of the same logic, now capable of drifting out of sync) is exactly the problem this lesson's structure avoids.

Level 2 - complete code:

```
// Level 2 is verification only - the Level 1 code is already correct, so
no lines change.
// Checks: after a win and after a loss, R restores 0/3/5 with no leftover
enemies; R during play does nothing.
```

Level 3 - code (additions highlighted):

Highlighted lines are the Level 3 additions on top of the Level 1 code.

```
// Level 3 visual: add a "switch costume" or "set color effect" block to
the Player's win/lose receive scripts.
when I receive [you win]
set [color v] effect to 50
say [You Win! Press R to Restart] for 3 seconds
set [color v] effect to 0

// BUG FIX (one good answer): an escape also resolves an enemy.
GameManager, "when I receive [enemy escaped]":
change lives by -1
change enemies_remaining by -1
if lives = 0 then
    set game_active to false
    broadcast [game over]
else
    if enemies_remaining = 0 then
        set game_active to false
        broadcast [you win]
    else
        start sound [sfx_lifelost]  // your Lesson 8 Level 3 sound
```

Common Student Mistakes

- Game-state drift: the student added start-up code or a game-over effect beyond the handout (extra green-flag scripts, an animation, a color change, a hidden object) and did not reset it on restart. Symptom: a restart that looks almost right. Walk through the Handout's "Audit Your Game State" list with them.
- Forgetting to remove the setup code from the green-flag script after moving it into "when I receive [start game]" - results in setup effectively running twice at game start (harmless here, but worth flagging as untidy duplication).
- Putting the stop-this-script check anywhere but the very top of the clone's forever loop, so some movement or escape blocks still run after game over.

- Trying to put "delete this clone" inside the spawn script (it is an end block, nothing can follow it) or forgetting the separate delete script: old clones also run the spawn loop on restart, multiplying enemies.
- Forgetting the game_active = false check on the restart key script, allowing R to interrupt an active round.